

Newton Raphson

- Equações do subsistema 1 a serem solucionadas

$$P_k^{\text{calculado}} = V_k \times \sum_{m=1}^n V_m \times (G_{km} \times \cos(\theta_{km}) + B_{km} \times \sin(\theta_{km})), \quad k \in \{PQ, PV\}$$

$$Q_k^{\text{calculado}} = V_k \times \sum_{m=1}^n V_m \times (G_{km} \times \sin(\theta_{km}) - B_{km} \times \cos(\theta_{km})), \quad k \in \{PQ\}$$

- Resíduos de Potência

$$\Delta P_k = P_k^{(\text{especificado})} - P_k^{(\text{calculado})} = 0, \quad k \in \{PQ, PV\}$$

$$\Delta Q_k = Q_k^{(\text{especificado})} - Q_k^{(\text{calculado})} = 0, \quad k \in \{PQ\}$$

- Sistema a ser solucionado pelo método de Newton-Raphson

$$\Delta P_k = 0; k \in PQ, PV$$

$$\Delta Q_k = 0; k \in PQ$$

- Considera-se o sistema com **n** barras, sendo que:
- **Barras PQ:** barras de 1 a l
- **Barras PV:** barras de l+1 a n-1
- **Barras Vθ:** barra n

$$J = - \begin{bmatrix} H & N \\ M & L \end{bmatrix}$$

$$H_{(n-1) \times (n-1)} = \begin{bmatrix} \frac{\partial P_1}{\partial \theta_1} & \dots & \frac{\partial P_1}{\partial \theta_{n-1}} \\ \vdots & \ddots & \vdots \\ \frac{\partial P_{n-1}}{\partial \theta_1} & \dots & \frac{\partial P_{n-1}}{\partial \theta_{n-1}} \end{bmatrix}; N_{(n-1) \times l} = \begin{bmatrix} \frac{\partial P_1}{\partial V_1} & \dots & \frac{\partial P_1}{\partial V_l} \\ \vdots & \ddots & \vdots \\ \frac{\partial P_{n-1}}{\partial V_1} & \dots & \frac{\partial P_{n-1}}{\partial V_l} \end{bmatrix}$$

$$M_{l \times (n-1)} = \begin{bmatrix} \frac{\partial Q_1}{\partial \theta_1} & \dots & \frac{\partial Q_1}{\partial \theta_{n-1}} \\ \vdots & \ddots & \vdots \\ \frac{\partial Q_l}{\partial \theta_1} & \dots & \frac{\partial Q_l}{\partial \theta_{n-1}} \end{bmatrix}; L_{(l \times l)} = \begin{bmatrix} \frac{\partial Q_1}{\partial V_1} & \dots & \frac{\partial Q_1}{\partial V_l} \\ \vdots & \ddots & \vdots \\ \frac{\partial Q_l}{\partial V_1} & \dots & \frac{\partial Q_l}{\partial V_l} \end{bmatrix}$$

$$H_{kk} = \frac{\partial P_k}{\partial \theta_k} = -V_k^2 B_{kk} - V_k \sum_{m \in \Omega_k} V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km})$$

$$H = \frac{\partial P(V, \theta)}{\partial \theta} \Rightarrow H_{km} = \frac{\partial P_k}{\partial \theta_m} = V_k V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km}) \quad m \in \Omega_k$$

$$H_{km} = 0 \quad m \notin \Omega_k$$

$$N_{kk} = \frac{\partial P_k}{\partial V_k} = V_k G_{kk} + \sum_{m \in \Omega_k} V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km})$$

$$N = \frac{\partial P(V, \theta)}{\partial V} \Rightarrow N_{km} = \frac{\partial P_k}{\partial V_m} = V_k (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km}) \quad m \in \Omega_k$$

$$N_{km} = 0 \quad m \notin \Omega_k$$

$$M_{kk} = \frac{\partial Q_k}{\partial \theta_k} = -V_k^2 G_{kk} + V_k \sum_{m \in \Omega_k} V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km})$$

$$M = \frac{\partial Q(V, \theta)}{\partial \theta} \Rightarrow M_{km} = \frac{\partial Q_k}{\partial \theta_m} = -V_k V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km}) \quad m \in \Omega_k$$

$$M_{km} = 0 \quad m \notin \Omega_k$$

$$L_{kk} = \frac{\partial Q_k}{\partial V_k} = -V_k B_{kk} + \sum_{m \in \Omega_k} V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km})$$

$$L = \frac{\partial Q(V, \theta)}{\partial V} \Rightarrow L_{km} = \frac{\partial Q_k}{\partial V_m} = V_k (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km}) \quad m \in \Omega_k$$

$$L_{km} = 0 \quad m \notin \Omega_k$$

Algoritmo

1. Ler os dados da rede
2. Formar a matriz de admitâncias **Y**.
3. Inicializar o contador de iterações: $i \leftarrow 0$.
4. Atribuir valores iniciais para as tensões: $V_k^{(i)} = V_k^0 = 1 + 0j$, para $k = 1, 2, \dots, n$.
5. Arbitrar condições iniciais das variáveis de estado : $(\theta^{(0)}, V^{(0)})$

6. Cálculo de $P_k^{\text{calculado}}, k \in [PQ, PV]$ e $Q_k^{\text{calculado}}, k \in [PQ]$

7. Calcular ΔP_k e ΔQ_k e verificar a convergência. $\text{Semax}[\Delta P_k] \leq \epsilon_p$ e $\text{max}[\Delta Q_k] \leq \epsilon_p$ parar
 $\Delta P_k = \Delta P_k^{\text{especificado}} - P_k^{\text{calculado}}, k \in [PQ, PV]$
 $\Delta Q_k = \Delta Q_k^{\text{especificado}} - Q_k^{\text{calculado}}, k \in [PQ]$

8. Incrementar o contador de iterações: $i \leftarrow i + 1$

9. Montar a matriz Jacobiana $J^{(i)}$

10. Solucionar o sistema linearizado

$$\begin{bmatrix} \Delta P \\ \Delta Q \end{bmatrix}^{(i)} = -J^{(i)} \times \begin{bmatrix} \Delta \theta \\ \Delta V \end{bmatrix}^{(i)}$$

11. Atualizar a solução do problema

$$\begin{bmatrix} \theta \\ V \end{bmatrix}^{(i+1)} = \begin{bmatrix} \theta \\ V \end{bmatrix}^{(i)} + \begin{bmatrix} \Delta \theta \\ \Delta V \end{bmatrix}^{(i)}$$

12. Voltar para o passo 5

Implementação

```
% ----- Exemplo Professor -----  
% Env Rec Rs[pu] Xs[pu] Ysh[pu] Rel_transf_tensao(a)[pu] Ang_def (rad)  
  
Dados_linha=[ 1 2 0.200 0.500 0.020 1 0;  
              1 3 0.100 0.500 0.015 1 0;  
              1 4 0.300 0.350 0.030 1 0;  
              2 4 0.300 0.150 0.040 1 0;  
              3 4 0.350 0.150 0.040 1 0;];  
  
% Barra Tipo V ̸ Pi Qi Pc Qc  
Dados_barra=[ 1 1 1.00 0 0.00 0.00 0.00 0.00; % Barra 1 tipo 1 → (V̸) – Barra slack  
              2 3 0.00 0 0.00 0.06 0.37 0.00; % Barra 2 tipo 3 → (PQ) – Barra de carga  
              3 2 0.97 0 0.90 0.00 0.00 0.00; % Barra 3 tipo 2 → (PV) – Barra de geração  
              4 2 1.00 0 0.80 0.00 0.00 0.00;]; % Barra 4 tipo 2 → (PV) – Barra de geração
```

1. Extrair os dados

```
% -----  
% (1) Dados da linha de transmissão  
% -----  
Orig = Dados_linha(:, 1);  
Dest = Dados_linha(:, 2);  
Rs = Dados_linha(:, 3);  
Xs = Dados_linha(:, 4);  
bsh = Dados_linha(:, 5);  
a = Dados_linha(:, 6);  
phi = Dados_linha(:, 7);  
NumLT = size(Orig, 1);  
  
% -----  
% (1) Dados da barra  
% -----  
Barra = Dados_barra(:, 1);  
Tipo = Dados_barra(:, 2);  
Vref = Dados_barra(:, 3);  
Theta = Dados_barra(:, 4);  
Pi = Dados_barra(:, 5);  
Qi = Dados_barra(:, 6);  
Pc = Dados_barra(:, 7);  
Qc = Dados_barra(:, 8);  
NumBarras = size(Dados_barra, 1);
```

2. Montagem da matriz Y

Simétrica $Y_{ij} = Y_{ji}$

```
% -----  
% (2) Montagem da matriz de admitâncias nodais (Y)  
% Considerando linhas com transformadores e admitâncias shunt  
% -----  
  
% Inicializa as matrizes de admitâncias  
Y = zeros(NumBarras, NumBarras); % Matriz de admitâncias série das linhas  
Ysh = zeros(NumBarras, NumBarras); % Matriz de admitâncias shunt (susceptâncias)  
  
% -----  
% Cálculo das admitâncias série das linhas  
% -----
```

```

for k = 1:NumLT
    % Admitância série da linha k (recíproco da impedância série)
    y_km = -1 / (Rs(k) + 1i * Xs(k));

    % Preenchimento dos termos fora da diagonal da matriz Y
    Y(Orig(k), Dest(k)) = -a(k) * exp(-1i * phi(k)) * y_km;
    Y(Dest(k), Orig(k)) = -a(k) * exp(1i * phi(k)) * y_km;
end

% -----
% Cálculo das admitâncias shunt das linhas
% -----
for k = 1:NumLT
    Ysh(Orig(k), Dest(k)) = -1i * bsh(k);
    Ysh(Dest(k), Orig(k)) = -1i * bsh(k);
end

% -----
% Montagem da matriz de admitâncias nodal completa (YBarra)
% -----
YBarra = -Y;

for k = 1:NumBarras
    % Soma das admitâncias de todas as conexões da barra k
    YBarra(k, k) = -(sum(Ysh(k, :)) - sum(Y(k, :)));
end
Y = YBarra

```

```

Y = 4x4 complex
    2.4860 - 5.2293i   -0.6897 + 1.7241i   -0.3846 + 1.9231i   -1.4118 + 1.6471i
   -0.6897 + 1.7241i    3.3563 - 2.9975i    0.0000 + 0.0000i   -2.6667 + 1.3333i
   -0.3846 + 1.9231i    0.0000 + 0.0000i    2.7984 - 2.9026i   -2.4138 + 1.0345i
   -1.4118 + 1.6471i   -2.6667 + 1.3333i   -2.4138 + 1.0345i    6.4922 - 3.9049i

```

```
G = real(Y)
```

```

G = 4x4
    2.4860   -0.6897   -0.3846   -1.4118
   -0.6897    3.3563         0   -2.6667
   -0.3846    0.0000    2.7984   -2.4138
   -1.4118   -2.6667   -2.4138    6.4922

```

```
B = imag(Y)
```

```

B = 4x4
   -5.2293    1.7241    1.9231    1.6471
    1.7241   -2.9975         0    1.3333
    1.9231         0   -2.9026    1.0345
    1.6471    1.3333    1.0345   -3.9049

```

3. Implementação do Algoritmo

```

% -----
% Cálculo do Fluxo de Potência pelo Método de Newton-Raphson
% -----

% Potências especificadas nas barras
Pesp = Pi - Pc; % Potência ativa líquida (geração - carga)
Qesp = Qi - Qc; % Potência reativa líquida (geração - carga)

%% Identificação dos tipos de barras
slack = find(Tipo == 1); % Barra Slack (referência) - tensão fixa (V e ângulo)
PV = find(Tipo == 2);    % Barras tipo PV - tensão e potência ativa fixas
PQ = find(Tipo == 3);    % Barras tipo PQ - potência ativa e reativa fixas
barrasPQePV = [PQ; PV];
barrasPQePV = sort(barrasPQePV);

%% Inicialização dos parâmetros
V = Vref;
V(Tipo==3) = 1;          % Barras PQ começam com V = 1 pu
Tol = 1e-5;              % Tolerância de convergência
max_iter = 100;          % Número máximo de iterações
iter = 0;                % Contador de iterações

%% Preparação dos índices
npq = length(PQ);        % Número de barras PQ == L
n = NumBarras;          % Número total de barras

%% Loop principal do método de Newton-Raphson
while true
    iter = iter + 1;

```

```

% -----
% (6) Potências calculadas
% -----
Pcalc = zeros(n,1);
Qcalc = zeros(n,1);
for k = 1:n
    for m = 1:n
        ang = Theta(k)-Theta(m);
        Pk = V(k)*V(m)*(G(k,m)*cos(ang) + B(k,m)*sin(ang));
        Pcalc(k) = Pcalc(k) + Pk;
        Qk = V(k)*V(m)*(G(k,m)*sin(ang) - B(k,m)*cos(ang));
        Qcalc(k) = Qcalc(k) + Qk;
        fprintf('Barra %d: Pk = %.4f pu | Qk = %.4f pu\n', k, Pk, Qk);
    end
    fprintf('Barra %d: Pcalc = %.4f pu | Qcalc = %.4f pu\n', k, Pcalc(k), Qcalc(k));
end

% -----
% (7) Cálculo dos desvios (mismatch)
% -----
dP = Pesp - Pcalc; % Desvio na potência ativa
dQ = Qesp - Qcalc; % Desvio na potência reativa
dQ = dQ(PQ);       % Apenas barras PQ possuem equações de Q

% Vetor de mismatch (primeiro P, depois Q)
mismatch = [dP(barrasPQePV); dQ];

% Critério de parada (tolerância ou máximo de iterações)
if max(abs(mismatch)) < Tol || iter >= max_iter

    %% -----
    % Solução do subsistema 2: Cálculo de P e Q
    %% -----
    fprintf('----- Solução do subsistema 2 ----- \n');
    for k = 1:n
        somaP = 0;
        somaQ = 0;
        for m = 1:n
            Gkm = G(k, m);
            Bkm = B(k, m);
            ang = Theta(k) - Theta(m);

            somaP = somaP + V(m) * (Gkm * cos(ang) + Bkm * sin(ang));
            somaQ = somaQ + V(m) * (Gkm * sin(ang) - Bkm * cos(ang));
        end
        Pcalc(k) = V(k) * somaP;
        Qcalc(k) = V(k) * somaQ;
        fprintf('Barra %d: Pcalc = %.4f pu | Qcalc = %.4f pu\n', k, Pcalc(k), Qcalc(k));
    end
    break
end

%% -----
% Montagem da matriz Jacobiana
% -----

% Blocos da Jacobiana
H = zeros(n-1, n-1); %  $\partial P/\partial \theta$  (n-1)x(n-1)
N = zeros(n-1, npq); %  $\partial P/\partial V$  (n-1)x(1)
M = zeros(npq, n-1); %  $\partial Q/\partial \theta$  (1)x(n-1)
L = zeros(npq, npq); %  $\partial Q/\partial V$  (1)x(1)

%% Cálculo de H ( $\partial P/\partial \theta$ )
for i = 1:(n-1)
    k = barrasPQePV(i); % Índice da barra k
    for j = 1:(n-1)
        m = barrasPQePV(j); % Índice da barra m
        if k == m
            % Elemento da diagonal H_kk
            soma = 0;
            for l = 1:n
                ang = Theta(k) - Theta(l);
                soma = soma + V(l)*(G(k,l)*sin(ang)-B(k,l)*cos(ang));
            end
            H(i,j) = -V(k)^2 * B(k,k) - V(k) * soma;
        else
            % Elemento fora da diagonal H_km
            ang = Theta(k) - Theta(m);
            H(i,j) = V(k) * V(m) * (G(k,m) * sin(ang) - B(k,m) * cos(ang));
        end
    end
end

```

```

        end
    end
end

%% Cálculo de N ( $\partial P / \partial V$ )
for i = 1:(n-1)
    k = barrasPQePV(i);
    for j = 1:npq
        m = PQ(j);
        if k == m
            soma = 0;
            for l = 1:n
                ang = Theta(k)-Theta(l);
                soma = soma + V(l)*(G(k,l)*cos(ang) + B(k,l)*sin(ang));
            end
            N(i,j) = V(k)*G(k,k) + soma;
        else
            ang = Theta(k)-Theta(m);
            N(i,j) = V(k)*(G(k,m)*cos(ang) + B(k,m)*sin(ang));
        end
    end
end

%% Cálculo de M ( $\partial Q / \partial \theta$ )
for i = 1:npq
    k = PQ(i);
    for j = 1:(n-1)
        m = barrasPQePV(j);
        if k == m
            soma = 0;
            for l = 1:n
                ang = Theta(k)-Theta(l);
                soma = soma + V(l)*(G(k,l)*cos(ang)+ B(k,l)*sin(ang));
            end
            M(i,j) = -V(k)^2*G(k,k) + V(k)*soma;
        else
            ang = Theta(k)-Theta(m);
            M(i,j) = -V(k)*V(m)*(G(k,m)*cos(ang) + B(k,m)*sin(ang));
        end
    end
end

%% Cálculo de L ( $\partial Q / \partial V$ )
for i = 1:npq
    k = PQ(i);
    for j = 1:npq
        m = PQ(j);
        if k == m
            soma = 0;
            for l = 1:n
                soma = soma + V(l)*(G(k,l)*sin(Theta(k)-Theta(l)) ...
                    - B(k,l)*cos(Theta(k)-Theta(l)));
            end
            L(i,j) = -V(k)*B(k,k) + soma;
        else
            L(i,j) = V(k)*(G(k,m)*sin(Theta(k)-Theta(m)) - B(k,m)*cos(Theta(k)-Theta(m)));
        end
    end
end

%% -----
% Resolução do sistema linear
%% -----
J = [H N; M L]; % Montagem da matriz Jacobiana completa
ThetaV = J\mismatch; % Solução do sistema linear

% Atualização das variáveis
dTheta = ThetaV(1:(n-1));
dV = ThetaV(n:end);

Theta(barrasPQePV) = Theta(barrasPQePV) + dTheta; % Atualiza ângulos
V(PQ) = V(PQ) + dV; % Atualiza magnitudes

%% -----
% Impressão dos resultados da iteração
%% -----

```

```

fprintf('----- %d iteração -----\n', iter);

for k = 1:NumBarras
    fprintf('Barra %d: V = %.4f ∠ %.4f rad\n', k, abs(V(k)), Theta(k));
end

% Desvio de P (barras PQ e PV)
for i = 1:npq
    barra = barrasPQePV(i);
    fprintf("ΔP_barra_%d = %.3f - (%.3f) = %.3f\n", barra, Pesp(barra), Pcalc(barra), dP(barra));
end

% Desvio de Q (apenas barras PQ)
for i = 1:length(PQ)
    barra = PQ(i);
    fprintf("ΔQ_barra_%d = %.3f - (%.3f) = %.3f\n", barra, Qesp(barra), Qcalc(barra), (Qesp(barra)- Qcalc(barra)));
end

fprintf('--- Matriz H (∂P/∂θ) ---\n');
disp(H);

fprintf('--- Matriz N (∂P/∂V) ---\n');
disp(N);

fprintf('--- Matriz M (∂Q/∂θ) ---\n');
disp(M);

fprintf('--- Matriz L (∂Q/∂V) ---\n');
disp(L);

fprintf('--- Jacobiana Completa ---\n');
disp(J);

fprintf('ΔV = %.6f | Δθ = %.6f rad\n', dV, dTheta);
end

```

Avaliação do método de newton

- Maior confiabilidade: converge para casos em que outros métodos divergem
- Número de iterações necessárias para a convergência independe da dimensão do problema
- Requer mais espaço de memória devido à matriz jacobiana
- tempo computacional por iteração é maior (Inverter a matriz jacobiana e multiplica-la por um vetor)
- Redução do espaço de memória dada por: 1. Armazenamento compacto, 2. Técnicas de fatoração (obter a inversa), 3. Ordenação ótima (ordem das linhas e colunas da jacobiana alterada para minimizar o número de operações durante a fatoração)
- Apresenta convergência quadrática
- Não é sensível a escolha da barra de referência
- É sensível a escolha do ponto inicial